

GuichetIA

Assistant administratif intelligent pour le Burkina Faso

Rapport Technique

Projet	Data Science 2026 — Option 7 : Assistant Juridique et Administratif
Groupe	OUEDRAOGO Zeïd El Gazeli, DICKO Mamoudou, OUEDRAOGO Abdoul Kader W. Ghislan
Classe	Master 1 Data Science IFOAD
Soutenance	Lundi 13 juillet 2026
GitHub	github.com/Odg75/projet_data_science_GuichetIA
Frontend	projet-data-science-guichet-ia.vercel.app
Backend	guichetia-backend.onrender.com

Table des matières

- 1. Introduction et contexte
- 2. Architecture du système RAG
- 3. Corpus et stratégie d'ingestion
- 4. Modèles retenus
- 5. Interface et déploiement
- 6. Évaluation
- 7. Limites et perspectives
- 8. Conclusion

1. Introduction et contexte

GuichetIA est un assistant administratif basé sur une architecture RAG (Retrieval-Augmented Generation) conçu pour répondre aux questions des citoyens du Burkina Faso sur les démarches administratives courantes. Le projet s'inscrit dans l'option 7 du sujet Projet Data Science 2026 proposé par le Dr Delwende D. Arthur Sawadogo dans le cadre du Master 1 Data Science IFOAD.

L'objectif central n'est pas de produire un chatbot conversationnel générique, mais un agent capable de rechercher l'information pertinente dans une base de connaissances locale et vérifiée, de formuler une réponse sourcée, et de reconnaître explicitement ses limites lorsqu'une question est hors de son périmètre.

1.1 Problématique

Les démarches administratives burkinabè (CNIB, passeport, création d'entreprise, casier judiciaire, acte de naissance, certificat de nationalité) sont documentées sur des portails officiels qui restent peu accessibles. GuichetIA propose un point d'entrée unique en langage naturel, répondant avec précision à partir de sources officielles vérifiées.

1.2 Périmètre couvert

Démarche	Informations couvertes	Source
CNIB	Pièces, coût (2 500 F CFA), délai, perte, renouvellement	oni.bf, police.gov.bf
Passeport	Pièces, coût (50 000 F CFA), délai 72h, renouvellement	oni.bf, police.gov.bf
Création SARL/EI	RCCM, IFU, CPC, CNSS, CEFORE/MEBF, délais	service-public.gov.bf
Casier judiciaire	Types, lieu de dépôt, délai, coût	service-public.gov.bf
Acte de naissance	Copies, jugement supplétif, état civil	service-public.gov.bf
Cert. nationalité	Pièces, demande, validité	service-public.gov.bf

Tableau 1 — Les six démarches couvertes par GuichetIA.

2. Architecture du système RAG

GuichetIA adopte une architecture RAG en deux phases distinctes : une phase hors ligne (ingestion et indexation) et une phase en ligne (interrogation, retrieval et génération de réponse).

[Phase hors ligne] PDFs sources → Extraction texte → Chunking → Embeddings → ChromaDB

[Phase en ligne] Question utilisateur → Embedding (HF API) → Recherche cosinus ChromaDB (top-6) → Construction contexte → LLM Groq/Llama-3.1-8B → Réponse + sources → Frontend

Figure 1 — Flux de données du pipeline RAG GuichetIA.

2.1 Composants principaux

Composant	Rôle	Outil
Ingestion	Extraction texte PDF, chunking, indexation	PyMuPDF + LangChain
Base vectorielle	Stockage et recherche cosinus	ChromaDB (persistant)
Embeddings	Vectorisation chunks et requêtes	HF Inference API
LLM	Génération de la réponse grounded	Groq / Llama-3.1-8B
Orchestration	Chaining retrieval + génération	LangChain
Backend API	Endpoints /health, /ask, /pdfs/{demarche}	FastAPI (Python)
Frontend	Interface de chat utilisateur	React + Tailwind CSS

Tableau 2 — Composants de l'architecture GuichetIA.

2.2 Prompt système et guardrails

Un SYSTEM_PROMPT strict impose trois règles : (1) répondre uniquement à partir du contexte récupéré, (2) retourner une formule de repli explicite si l'information est absente, (3) ne jamais citer d'URL dans la réponse — l'interface affiche les sources automatiquement.

3. Corpus et stratégie d'ingestion

3.1 Construction du corpus

Aucun PDF officiel unique ne couvre intégralement chaque démarche. Le contenu a été vérifié manuellement sur les portails officiels burkinabè, puis compilé en 6 PDFs sources versionnés dans le dépôt git (backend/data/pdfs/) pour garantir la reproductibilité de l'indexation.

Fichier PDF	Démarche	Source officielle
cnib.pdf	CNIB	oni.bf + police.gov.bf
passport.pdf	Passeport	oni.bf + police.gov.bf
creation_entreprise.pdf	Création d'entreprise	service-public.gov.bf (PP + PM)
casier_judiciaire.pdf	Casier judiciaire	service-public.gov.bf
acte_naissance.pdf	Acte de naissance	service-public.gov.bf
certificat_nationalite.pdf	Cert. de nationalité	service-public.gov.bf

Tableau 3 — Les 6 PDFs sources du corpus GuichetIA.

3.2 Pipeline d'ingestion

- **Extraction** : PyMuPDF extrait le texte brut de chaque PDF. Les métadonnées (démarche) sont préservées.
- **Chunking** : RecursiveCharacterTextSplitter — 500 caractères, recouvrement 50. Résultat : 32 chunks.
- **Embeddings** : Vectorisation via HF Inference API (paraphrase-multilingual-MiniLM-L12-v2). Vecteurs normalisés (norme L2 = 1) avant indexation.
- **Indexation** : ChromaDB persistant avec métrique cosinus (hnsw:space=cosine).

Lors de l'interrogation, la question est vectorisée de la même façon, puis une recherche **cosinus** retourne les top-6 chunks. Score : **(1 – distance) × 100**.

4. Modèles retenus

4.1 Modèle d'embeddings

Attribut	Valeur
Modèle	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Fournisseur	HuggingFace (Inference API — gratuit)
Dimension	384
Langues	50+ langues dont français
Justification	Multilingue, léger, gratuit, bon français administratif

Tableau 4 — Caractéristiques du modèle d'embeddings.

4.2 LLM de génération

Attribut	Valeur
Modèle	llama-3.1-8b-instant
Fournisseur	Groq (API gratuite, LPU très rapide)
Température	0.1 — réponses factuelles et reproductibles
Latence typique	500–1500 ms génération 300–800 ms retrieval
Justification	Gratuit, rapide, performant en français

Tableau 5 — Caractéristiques du LLM de génération.

5. Interface et déploiement

5.1 Backend (FastAPI)

- **GET /health** : état de l'index ChromaDB et du LLM.
- **POST /ask** : exécute le pipeline RAG, retourne réponse + chunks + scores cosinus + sources.
- **GET /pdfs/{démarche}** : sert le PDF source officiel de la démarche.

5.2 Frontend (React + Tailwind)

- **Landing page** : présentation, statistiques (6 démarches, 32 chunks, 100 % retrieval), accès aux 6 démarches.
- **Chat Dashboard** : conversation avec suggestions contextuelles, badges sources, indicateurs de fiabilité (bouclier vert = réponse RAG, alerte orange = hors périmètre), liens vers sites officiels (oni.bf, police.gov.bf...).
- **Panneau RAG** : métriques en temps réel (chunks récupérés, temps de recherche, temps de génération, scores de pertinence cosinus, modèle LLM utilisé).
- **Page À propos** : description de la stack et de l'architecture.

5.3 Déploiement

Composant	Plateforme	URL	Plan
Backend	Render	guichetia-backend.onrender.com	Gratuit (512 Mo RAM)
Frontend	Vercel	projet-data-science-guichet-ia.vercel.app	Gratuit
Code source	GitHub	github.com/Odg75/projet_data_science_GuichetIA	Public

Tableau 6 — Plateformes de déploiement.

5.4 Captures d'écran de l'interface

Les figures ci-dessous illustrent les trois vues principales de l'application déployée.



Figure 2 — Page d'accueil : titre, badge RAG opérationnel, statistiques (6 démarches, 32 chunks, 100 %), cartes des démarches.

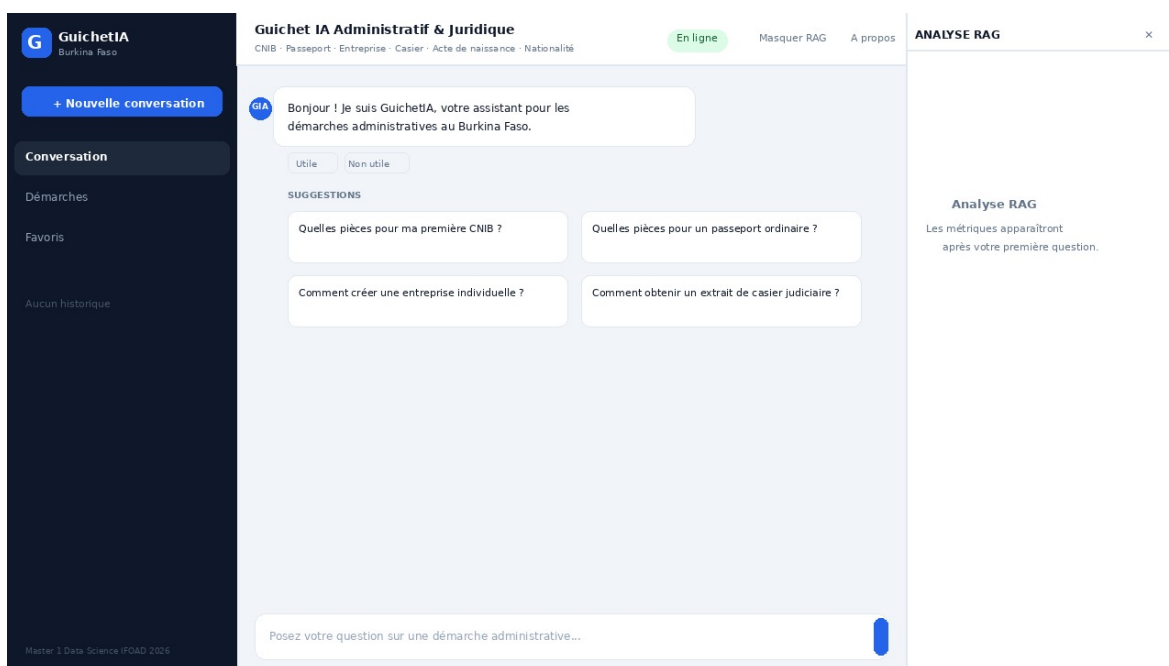


Figure 3 — Interface de chat : message de bienvenue, suggestions de questions et panneau d'analyse RAG.

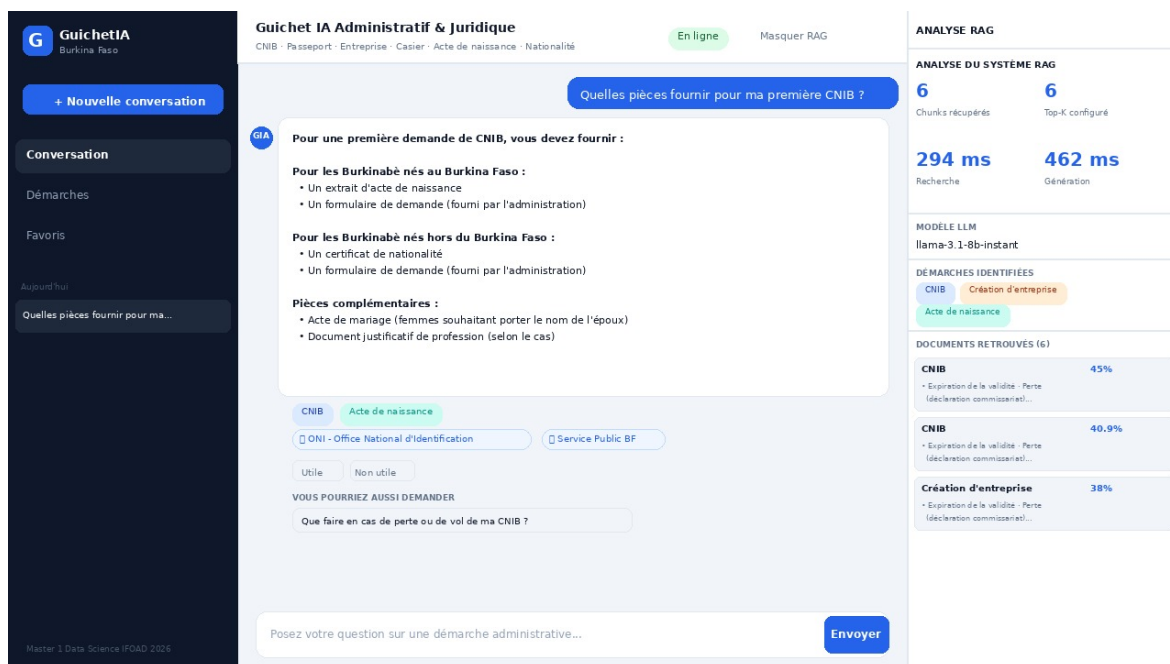


Figure 4 — Réponse à « Quelles pièces fournir pour ma première CNIB ? » avec badges sources, liens officiels et métriques RAG (6 chunks, 294 ms recherche, 462 ms génération).

6. Évaluation

L'évaluation est réalisée via `backend/evaluation/evaluate.py` sur 10 questions couvrant les 6 démarches et 2 questions hors périmètre.

6.1 Jeu de test

Paramètre	Valeur
Questions totales	10
Questions in-scope	8 (une ou deux par démarche)
Questions out-scope	2 (hors périmètre — visa, recette)
Script	<code>backend/evaluation/evaluate.py</code>
Résultats	<code>backend/evaluation/results.json</code>

Tableau 7 — Paramètres du jeu d'évaluation.

6.2 Résultats

Métrique	Résultat	Détail
Pertinence du retrieval (in-scope)	100 %	8/8 : démarche attendue présente dans les top-6 chunks récupérés
Non-hallucination (out-scope)	100 %	2/2 questions hors périmètre : formule de repli retournée, aucune invention
Score de pertinence	Métrique cosinus	$(1 - \text{dist}) \times 100$ — 0 = aucune similarité, 100 = identique sémantiquement

Tableau 8 — Résultats de l'évaluation.

Le taux de 100 % sur les deux métriques est robuste sur ce corpus de 32 chunks. Il devra être réévalué lors d'une extension à de nouvelles démarches.

7. Limites et perspectives

7.1 Limites actuelles

- **Corpus restreint** : 6 démarches, 32 chunks. Extension manuelle requise pour chaque nouvelle démarche.
- **Dépendance aux sources** : toute modification officielle (tarifs, procédures) nécessite mise à jour des PDFs et réindexation.
- **Modèle généraliste** : paraphrase-multilingual-MiniLM-L12-v2 n'est pas spécialisé en droit administratif burkinabè.
- **Cold start Render** : redémarrage après inactivité (~30–60 s). Contrainte du plan gratuit.
- **Jeu d'évaluation limité** : 10 questions ne constituent pas une évaluation statistiquement robuste.

7.2 Perspectives d'amélioration

- **Extension du corpus** : permis de conduire, mariage civil, permis de construire — même pipeline, sans modification architecturale.
- **Re-ranking** : cross-encoder après retrieval initial pour améliorer la sélection des chunks.
- **Évaluation RAGAS** : faithfulness, answer relevance, context precision sur 50+ questions annotées.
- **Multimodalité** : soumission de photo de document (OCR + classification démarche).
- **Déploiement offline** : modèle local quantifié (Ollama + Llama.cpp) pour fonctionnement sans connexion.

8. Conclusion

GuichetIA constitue une implémentation fonctionnelle et déployée d'un système RAG appliqué aux démarches administratives burkinabè. Le pipeline couvre 6 démarches, 6 PDFs sources et 32 chunks indexés, avec un taux de pertinence du retrieval de 100 % et un taux de non-hallucination de 100 % sur le jeu de test.

L'architecture — ChromaDB (cosinus) + HuggingFace Inference API + Groq/Llama 3.1 + FastAPI + React — est entièrement gratuite, reproductible par script, et déployée publiquement sur Render et Vercel.

La structure modulaire (ingestion séparée, backend séparé du frontend) et les pistes d'amélioration identifiées (re-ranking, RAGAS, extension corpus) permettent d'envisager une version production robuste sans refonte architecturale majeure.